Modular, Adaptive, Policy-based Autoscaling For Kubernetes

A case study on the reCluster architecture

Thesis defense, Master in Computer Science
University of Trento,
Department of Information engineering and computer science
15/10/2025

Supervisor : Lorenzo Angeli

Student : Lorenzo Cereser

Social

Universities were pioneers in deploying their own digital infrastructure. Now they have lost their capabilities in favor of outsourcing to external actors.

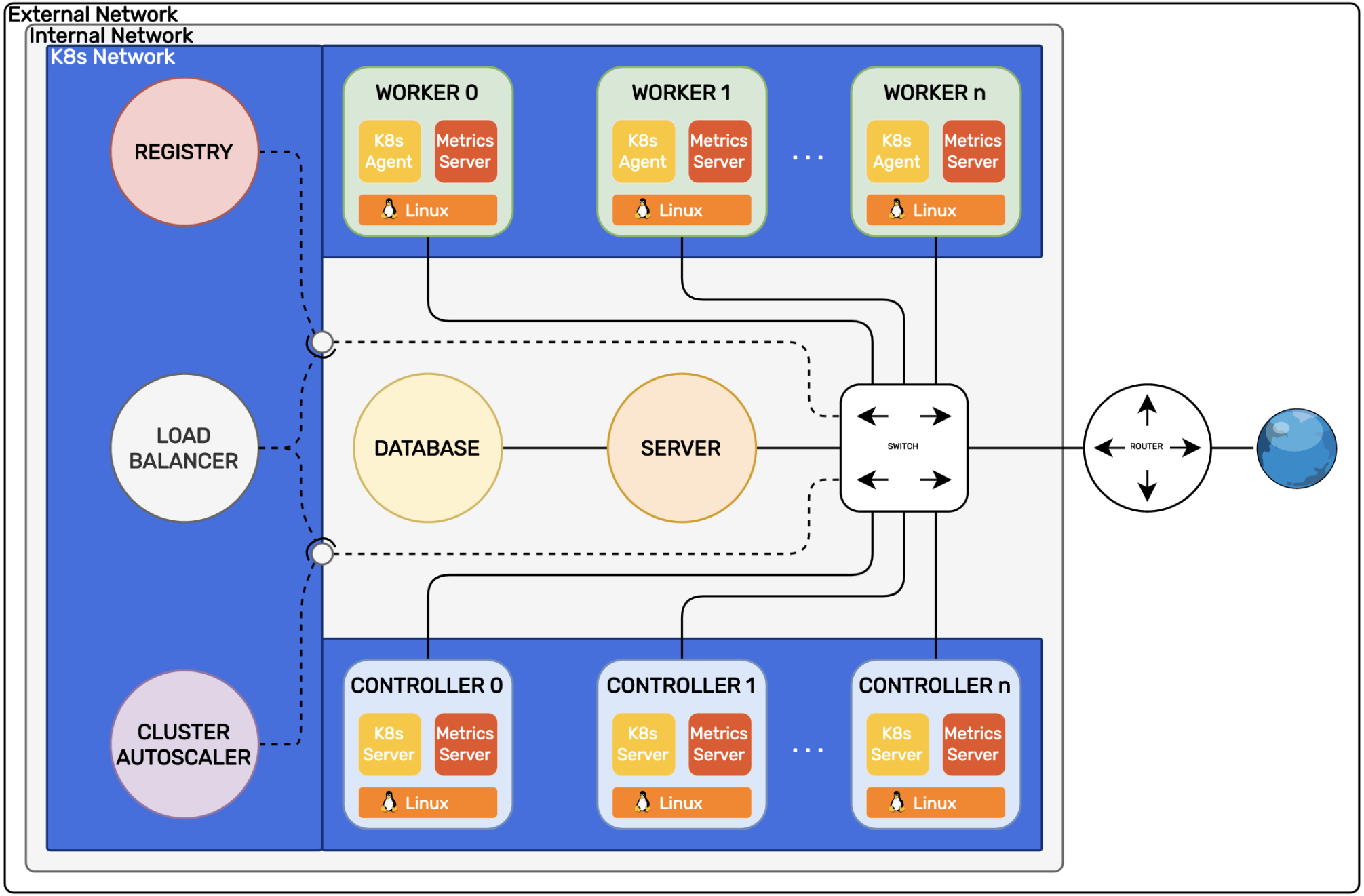
Empirical

Vast amounts of institutional hardware (laptops, devboards, etc.) is currently sitting unused.

ReCluster project

Cluster of upcycled hardware to run institutional infrastructure. ReCluster focus is on energy efficiency.

Legacy reCluster Architecture

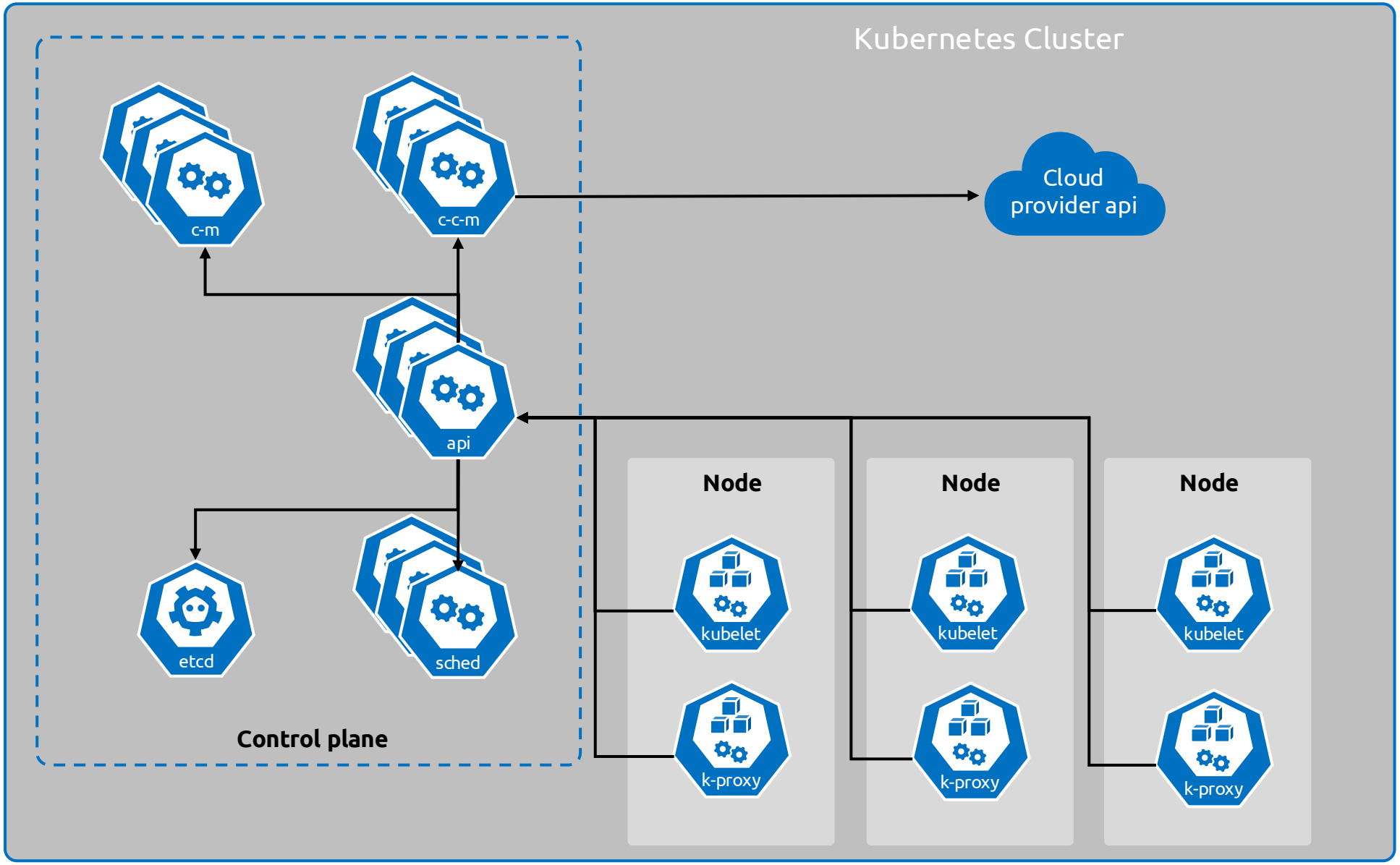


Picture credits : Carlo Corradini, 2022 ReCluster A resource-aware Kubernetes architecture for heterogeneous clusters

kubernetes

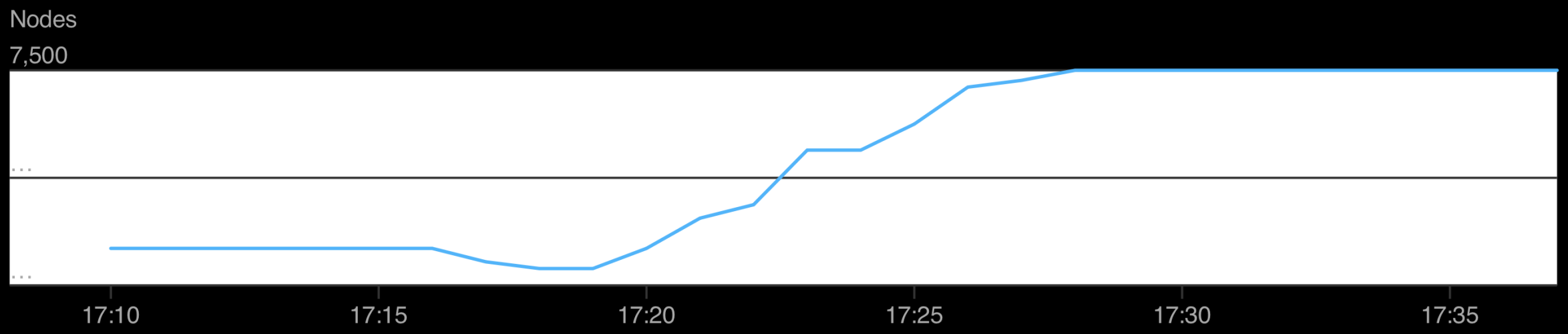
Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services.

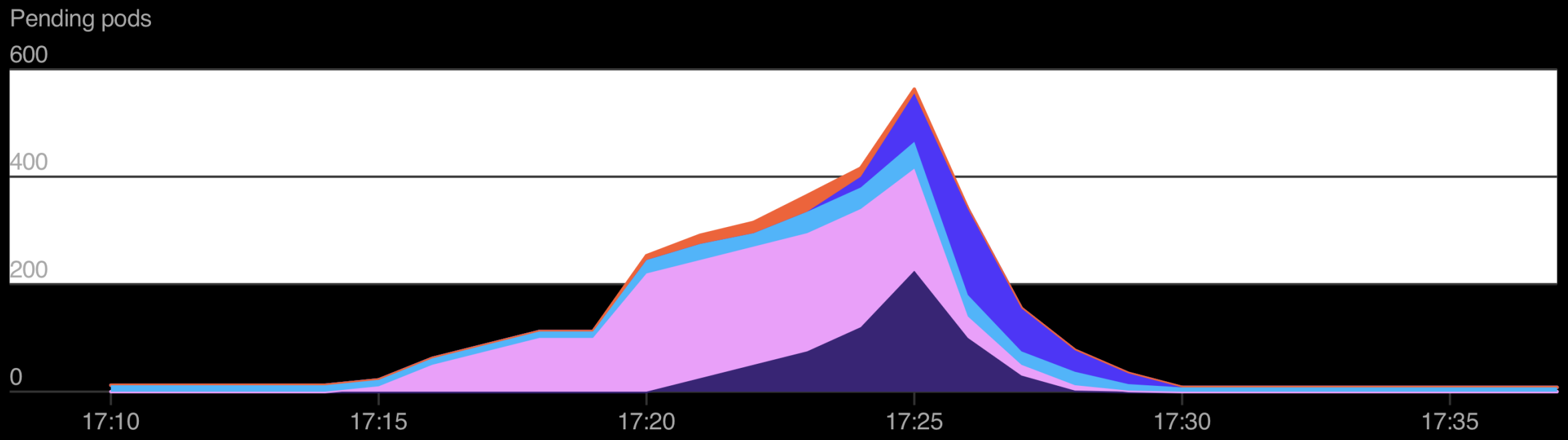
Picture credits : https://1000logos.net/kubernetes-logo/



- **Cluster**
- **Pods**
- **Nodes**
- Namespaces
- Control plane
- Workloads
- Ingress
- WebHooks
- Service
- …

Image adapted from https://kubernetes.io/docs/concepts/architecture/





Credits: OpenAI, 'Scaling Kubernetes to 7,500 nodes' (Jan 25, 2021), https://openai.com/index/scaling-kubernetes-to-7500-nodes/

El Haj Ahmed, Ghofrane & Gil-Castiñeira, Felipe & Costa-Montenegro, Enrique. (2020). KubCG: **“A dynamic Kubernetes scheduler for heterogeneous clusters. Software: Practice and Experience.”** 51. 213-234. 10.1002/spe.2898.

S. Yang, Y. Ren, J. Zhang, J. Guan and B. Li, "**KubeHICE: Performance-aware Container Orchestration on Heterogeneous-ISA Architectures in Cloud-Edge Platforms**" 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking.

H. Albahar, S. Dongare, Y. Du, N. Zhao, A. K. Paul and A. R. Butt, **"SchedTune: A Heterogeneity-Aware GPU Scheduler for Deep Learning"** 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Taormina, Italy, 2022, pp. 695-705, doi: 10.1109/CCGrid54584.2022.00079.

Decker, J.; Hasse, V.F.; Kunkel, J. Q8S: **“Emulation of Heterogeneous Kubernetes Clusters Using QEMU”**. *Algorithms* 2025, *18*, 324. https://doi.org/10.3390/a18060324

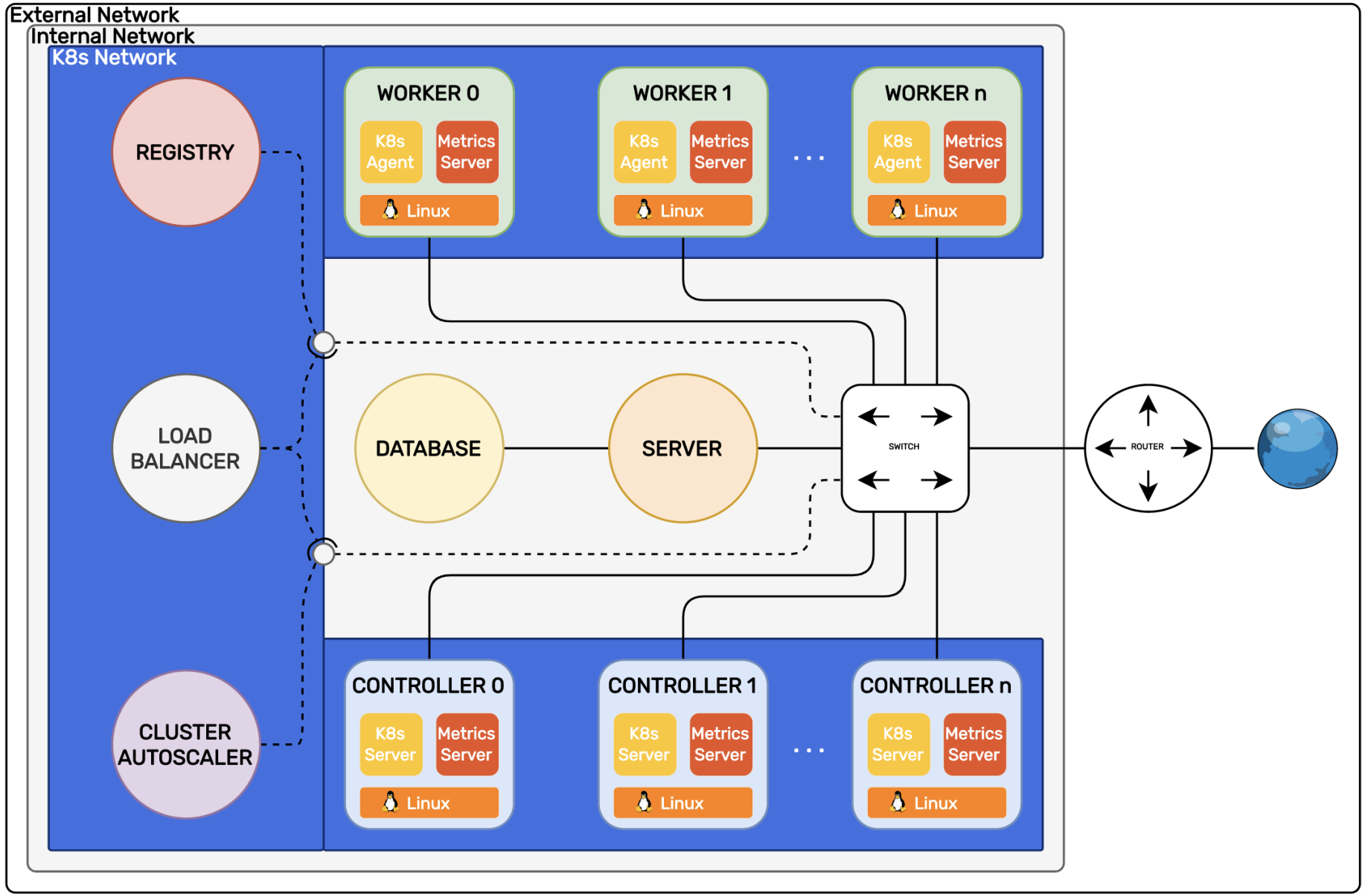
- Upcycled and resource-constrained environments
- Sustainability-driven infrastructures
- Edge and far edge sites
- Emergency infrastructures

Why now ?

Mainstream availability of energy-efficient server class ARM processors.

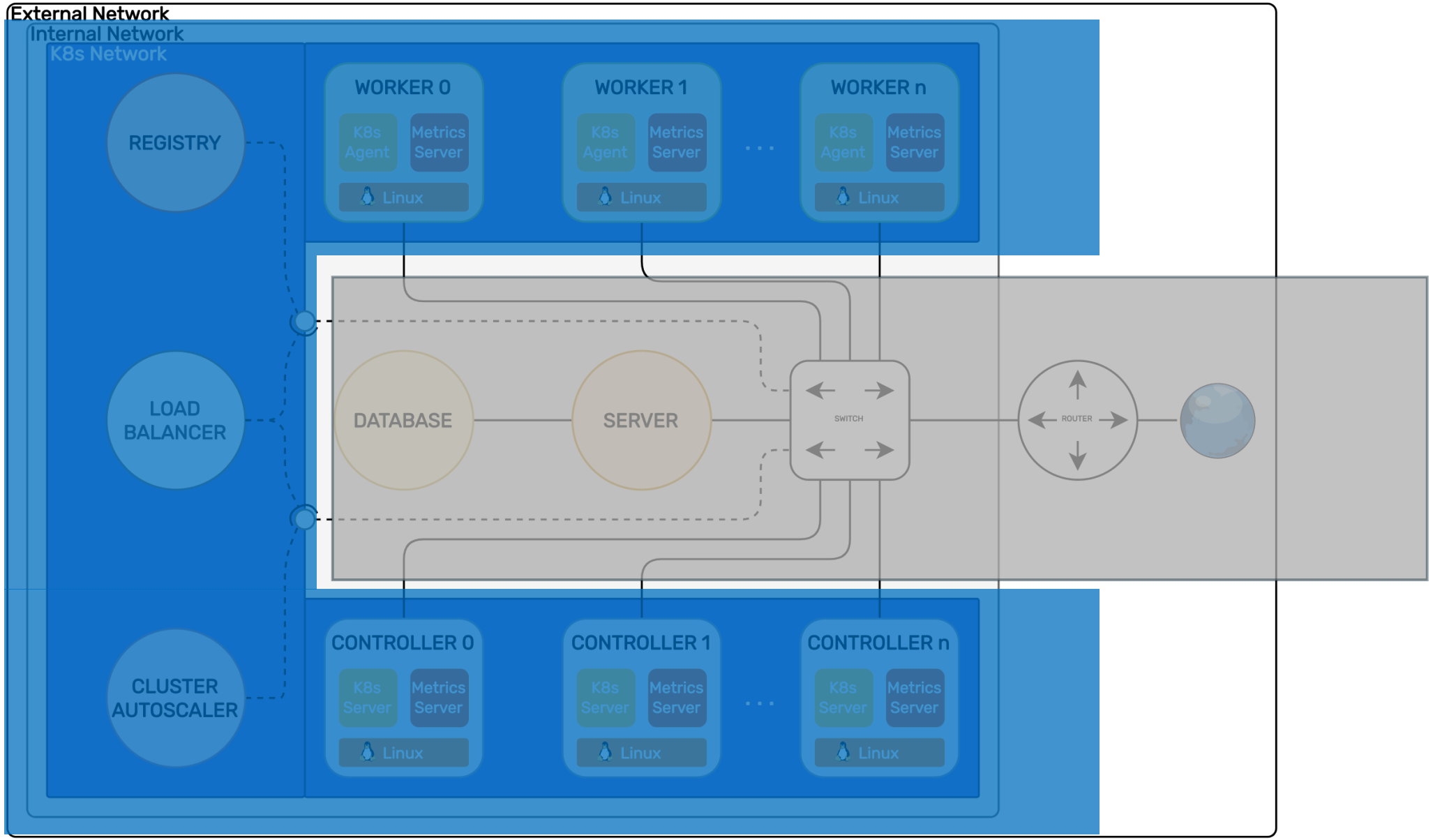
Maturity of Kubernetes extensibility

The Instability of the socio-political landscape, which increases the value of adaptive orchestration.



- Out of cluster management of cluster state
- More components to be maintained
- No reasoning about cluster operator priorities

Picture credits: Carlo Corradini, 2022 ReCluster A resource-aware Kubernetes architecture for heterogeneous clusters



Picture adapted from : Carlo Corradini, 2022 ReCluster A resource-aware Kubernetes architecture for heterogeneous clusters

1st Contribution

Kubernetes-Native reCluster design

The platform redesign is aimed at providing a Kubernetes native reCluster solution for highly heterogeneous cluster.

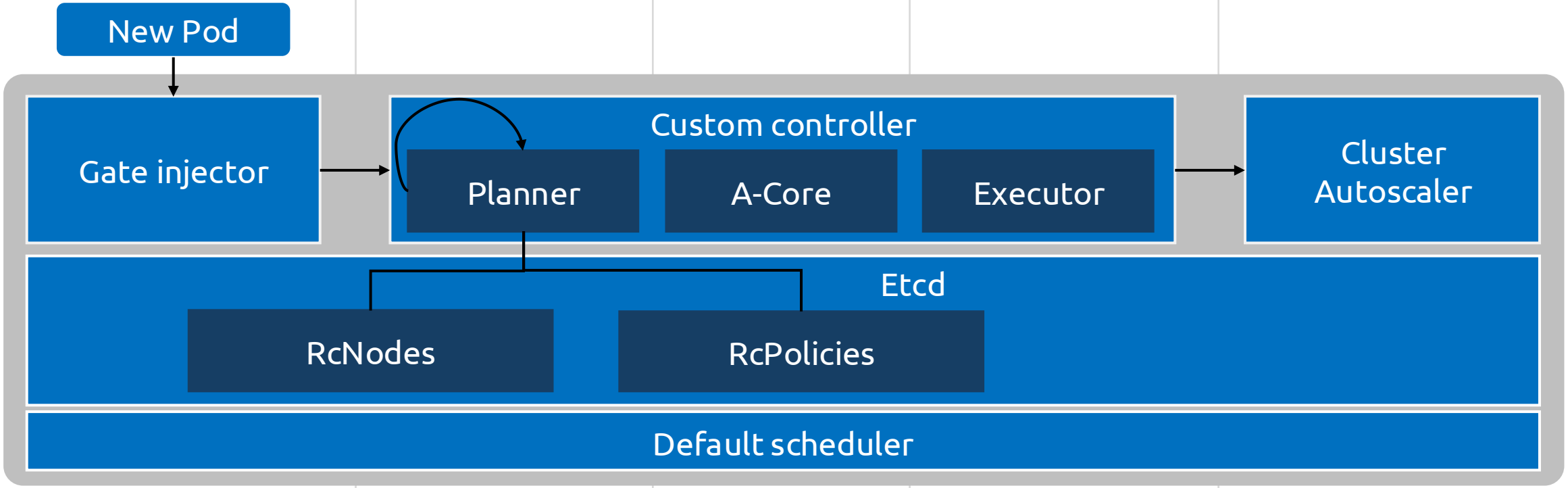
RcNode resource
In-cluster declarative inventory of all nodes available, with precise hardware capabilities.

RcPolicy resource
Representation as a custom resource of operator intent.

Custom controller
Governs from planning and nodes to pods binding.

Gate Injector
Injects scheduling gates into pending pods, allowing the custom controller to work properly.

Action Loop



Prevents the newly arrived pods to be scheduled by the default scheduler, implemented with a K8s mutating WebHook

Periodically checks rcNodes and policies as well as cluster states

Translates rcnodes and rcpolicies into an optimized execution plan

Implements the plan's actions inside the cluster

Deals with cluster expansion or contraption matters in synergy with executor

Algorithmic core

	Filter (Feasibility and Safety)	Score (Normalized Multi-Objective)	Solve (Set Selection under Budgets and Conflicts)
Inputs	Pending pods, nodes, policies, safety budgets	Admissible pod-node pairs or node actions	Scored candidates with values, costs and conflicts
Description	Prunes infeasible nodes based on resources, policy gates, safety limits, and boot reachability	Computes features (e.g., packing, performance, energy), normalizes, weights by policy, subtracts risks/energy costs.	Maximizes total score of selected actions under budgets and no conflicts Regime Switching: • Exact for small sizes of sets • Heuristic: For medium sizes of sets • Greedy: For big sizes of sets
Output	Reduced candidate set for binding/migration or node actions	Single score per admissible action	Plan of actions (e.g., bind, power up) with temporal constraints

2nd Contribution

Virtual testbed and experiments

- **Virtual testbed**

Virtual one-command-deploy of a virtual testbed.

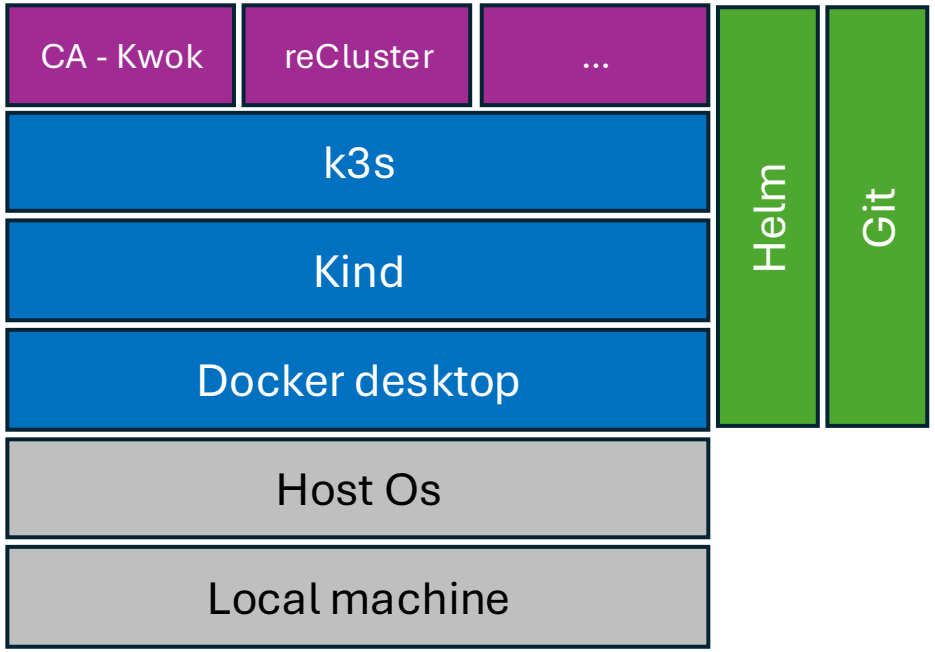
- **Comprehensive testing framework**

Including node groups of different sizes and ablation tests.

! The virtual testbed testing is not to be considered a substitute of bare-metal testing, it is instead a necessary preliminary step to rule out any logical issues with the platform redesign and implementation.

Virtual Testbed

- CA-Kwok: Cluster autoscaler configured with KWOK for emulating node behaviors (e.g., boot delays, power states).
- reCluster: Extended framework with policy-based scoring and scheduling.
- k3s: Lightweight Kubernetes distribution.
- Kind: Kubernetes IN Docker for local clusters.
- Docker Desktop: Container runtime.
- Host OS Local Machine: Base system (e.g., M3 Pro-like hardware).
- Helm Git: Deployment and version control tools.



Evaluation & Methodology – Nodes and groups

Sentinel nodes

- Amd64
- 4 cores
- 8 GiB
- boot p50 8 s
- idle 22 W
- active 40 to 70 W

Balanced nodes

- Amd64
- 16 cores
- 32 GiB
- boot p50 20 s
- idle 35 W
- active 70 to 140 W

Efficient nodes

- Arm64
- 32 cores
- 64 GiB
- boot p50 25 s
- idle 18 W
- active 45 to 95 W

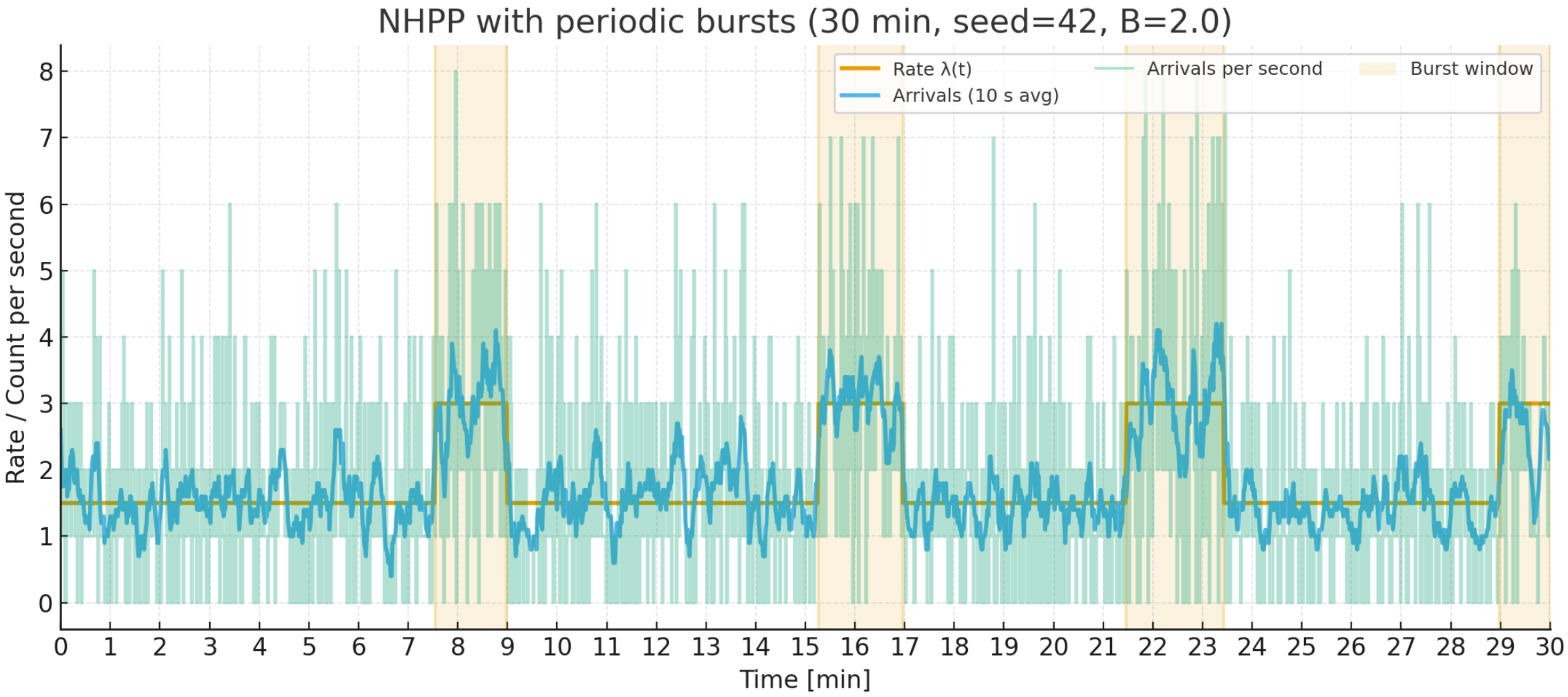
Throughput nodes

- Amd64
- 48 cores
- 128 GiB
- boot p50 55 s
- idle 65 W
- active 140 to 260 W

B0	stock upstream
G1	3x balanced nodes
G2	1x balanced, 1x efficient, 1x sentinel
G3	1x throughput, 2x efficient

G4	10x efficient, 6x balanced, 4x Sentinel
G5	50x efficient, 30x balanced, 20x sentinel
G6	500x efficient, 300x balanced, 200x Sentinel

Evaluation & Methodology - Workloads



Results - Scheduling and readiness

Group	T_{gate} ms	$T^{\text{abs}}_{\text{sched}}$ ms	$T^{\text{abs}}_{\text{ready}}$ ms	Migrations per hour
B0	0 (0)	16 (37)	395 (540)	0.3
G1	0 (0)	18 (41)	410 (560)	0.6
G2	12 (35)	24 (55)	520 (740)	1.1
G3	5 (18)	22 (49)	460 (650)	3.0
G4	8 (22)	25 (57)	510 (720)	2.2
G5	15 (40)	31 (68)	590 (860)	4.7
G6	18 (55)	38 (79)	640 (940)	6.3

The results show that the reCluster system introduces **minimal overhead** in **scheduling latency**, with gate waits (Tgate) remaining low (under 20 ms median even in large-scale G6) despite handling heterogeneous nodes and activations.

Absolute readiness times (T abs ready) increase modestly in heterogeneous setups (e.g., 520 ms in G2 vs. 395 ms in B0) due to boot delays and policy-guided placements, but this is offset by better resource efficiency.

Results - packing efficency

Group	Packing efficiency %	Active nodes	Convergence time s
B0	84.7 (88.3)	3 (3)	52
G1	88.9 (92.1)	3 (3)	46
G2	90.4 (93.0)	3 (4)	72
G3	92.1 (94.2)	3 (4)	64
G4	93.8 (95.1)	14 (16)	120
G5	91.7 (93.6)	71 (78)	240
G6	89.6 (91.5)	645 (702)	410

Packing efficiency consistently improves with reCluster's policy exploitation, rising from 84.7% in stock baseline B0 to over 93% in optimized G4.

Active node counts remain stable (e.g., 3-4 in small groups), **but convergence time increases with scale** indicating the system needs more time to stabilize large fleets yet it stays within practical limits.

The **dip in efficiency** for larger groups (G5-G6) suggests heuristic approximations trade some optimality for speed.

Results - Estimated energy

Group	Energy per request J	Idle share %	Activation energy J	CO_2e g
B0	13.9 (15.8)	31.2 (36.5)	0 (0)	191 (221)
G1	12.8 (14.6)	28.5 (33.2)	0 (0)	182 (206)
G2	11.3 (13.1)	24.1 (28.7)	620 (910)	169 (195)
G3	10.6 (12.2)	21.8 (26.9)	710 (980)	161 (186)
G4	9.9 (11.4)	20.2 (24.8)	2,300 (3,100)	152 (177)
G5	10.7 (12.5)	22.6 (27.9)	8,900 (12,400)	164 (193)
G6	11.9 (13.8)	26.4 (31.5)	74,000 (95,000)	179 (212)

Results show achieves **notable energy savings**, reducing energy per request from **13.9 J in B0 to as low as 9.9 J in G4** by consolidating onto efficient nodes and amortizing activation costs during steady periods.

Activation energy rises with scale (up to 74,000 J in G6) due to more power-ons, but this is **balanced by overall CO2e reductions (e.g., 152 g in G4 vs. 191 g in B0)**, assuming constant emissions factors.

Key Limitations & tradeoffs

- Virtual testbed limitation
- Workload Approximation
- Scope Exclusions

Future work

- Physical Deployment and Validation
- Enhanced Measurement and Modeling
- Hardware Scope Broadening
- Policy and Optimization Extensions

Thank you

Questions

Main contributions:

- K8s native redesign
- Virtual testbed
- Testing framework

Estimated energy and carbon formula

We estimate total energy as the sum of steady draw integrated over time and discrete activation pulses, then convert energy to a carbon proxy using a constant emissions factor,

$$E_{\text{total}} = \sum_{i \in \mathcal{N}} \int_0^T P_i\big(u_i(t)\big)\, dt \; + \sum_{a \in \mathcal{A}_{\text{act}}} E_{\text{act}}(a), \qquad \text{CO}_{2\text{e}} = \kappa \cdot \frac{E_{\text{total}}}{3600}.$$

- $P_i(u)$ in W, node i power at utilization u in $[0, 1]$, with $P_i(u) = W_i^{\text{idle}} + u\,(W_i^{\max} - W_i^{\text{idle}})$ unless noted.
- $u_i(t)$, instantaneous CPU utilization of node i.
- $E_{\text{act}}(a)$ in J, energy of activation event a, for example boot pulse.
- κ in $\text{gCO}_2\text{e}\,\text{kWh}^{-1}$, emissions factor, division by 3600 converts J to Wh.